Name of Student: Vyankatesh Rohokale	PRN: 26070122518
Semester: III	Year: AY 2026-27
Subject: Programming with Java	
ASSIGNMENT No: 26	
TITLE: Write a Java program to create and execute multiple threads using the Runnable	

Problem Statement

Write a Java program to create and execute multiple threads using the Runnable interface.

Learning Objectives

To create multiple threads using the Runnable interface.

Programme

1. Create a program using Runnable to run two tasks simultaneously.

```
class Task1 implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Task 1: " + i);
        }
    }
}
class Task2 implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Task 2: " + i);
        }
    }
}

public class RunnableDemo {
    public static void main(String[] args) {
        Thread t1 = new Thread(new Task1());
        Thread t2 = new Thread(new Task2());

        t1.start();
        t2.start();
    }
}
```

```
piyushrohokale@Piyushs-MacBook-Air assignment26 % javac RunnableDemo.java
piyushrohokale@Piyushs-MacBook-Air assignment26 % java RunnableDemo
Task 1: 1
Task 1: 2
Task 2: 1
Task 1: 3
Task 2: 2
Task 2: 3
Task 2: 4
Task 1: 4
Task 2: 5
Task 1: 5
piyushrohokale@Piyushs-MacBook-Air assignment26 %
```

2. Create a Java program using the Runnable interface where two tasks run simultaneously: one thread performs deposit operations, and another performs withdrawal operations, both displaying results concurrently.

```
class BankAccount {
    int balance = 10000;

    synchronized void deposit(int amount) {
        balance += amount;
        System.out.println("Deposited: " + amount + " |
Balance: " + balance);
    }

    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount + " |
Balance: " + balance);
        } else {
            System.out.println("Insufficient balance");
        }
    }
}

class DepositTask implements Runnable {
    BankAccount account;

    DepositTask(BankAccount account) {
        this.account = account;
    }
```

```
    public void run() {
        for (int i = 0; i < 5; i++) {
            account.deposit(500);
        }
    }
}

class WithdrawTask implements Runnable {
    BankAccount account;

    WithdrawTask(BankAccount account) {
        this.account = account;
    }

    public void run() {
        for (int i = 0; i < 5; i++) {
            account.withdraw(300);
        }
    }
}

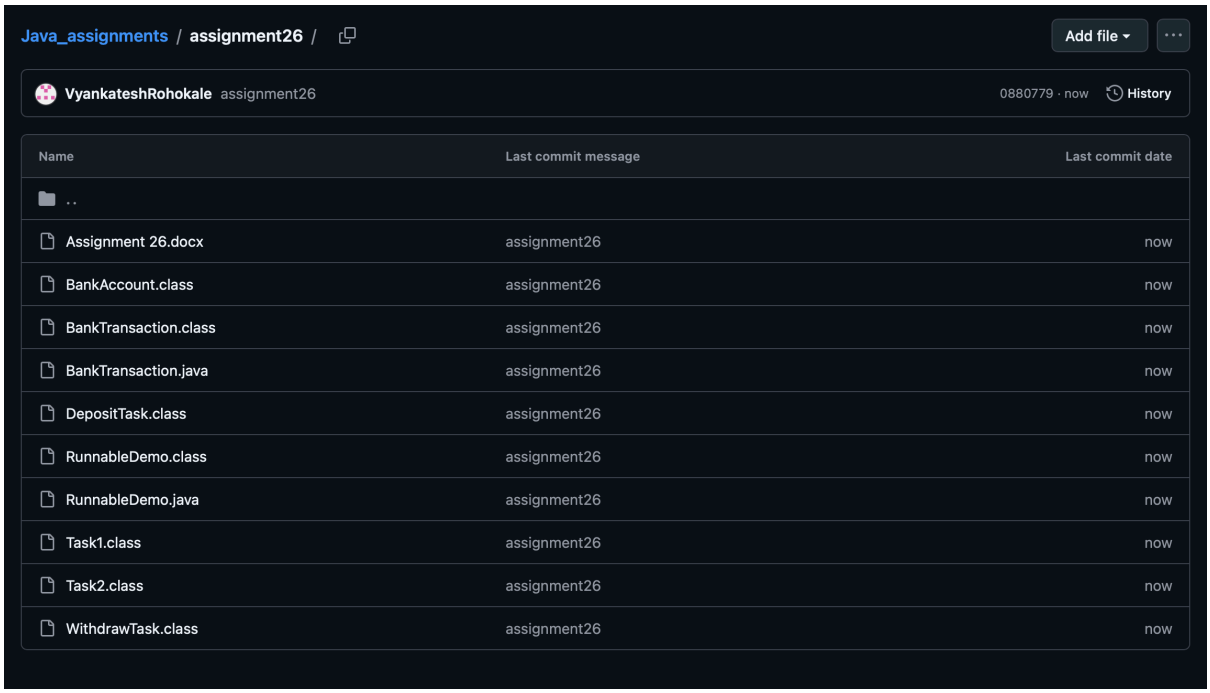
public class BankTransaction {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        Thread t1 = new Thread(new DepositTask(account));
        Thread t2 = new Thread(new WithdrawTask(account));

        t1.start();
        t2.start();
    }
}
```

```
piyushrohokale@Piyushs-MacBook-Air assignment26 % java BankTransaction
Deposited: 500 | Balance: 10500
Deposited: 500 | Balance: 11000
Deposited: 500 | Balance: 11500
Deposited: 500 | Balance: 12000
Deposited: 500 | Balance: 12500
Withdrawn: 300 | Balance: 12200
Withdrawn: 300 | Balance: 11900
Withdrawn: 300 | Balance: 11600
Withdrawn: 300 | Balance: 11300
Withdrawn: 300 | Balance: 11000
piyushrohokale@Piyushs-MacBook-Air assignment26 %
```

Github:



Conclusion:

The first program demonstrates two tasks executing concurrently, while the second simulates bank deposit and withdrawal operations using two threads that share a bank account. The synchronized keyword ensures that only one thread at a time modifies the shared balance through these methods.